60. Псевдонимы типов: typedef и type alias

На этом уроке мы рассмотрим псевдонимы типов typedef и type alias в языке C++.

Оглавление:

1. typedef
2. typedef и читабельность кода
3. typedef и поддержка кода
4. typedef и кроссплатформенность
5. typedef и упрощение сложного
6. type alias
7. Тест

typedef

Ключевое слово typedef позволяет программисту создать псевдоним для любого типа данных и использовать его вместо фактического имени типа. Чтобы объявить typedef (использовать псевдоним типа) — используйте ключевое слово typedef вместе с типом данных, для которого создается псевдоним, а затем, собственно, сам псевдоним. Например:

```
typedef double time_t; // используем time_t в качестве псевдонима для типа double

// Следующие два стейтмента эквивалентны
double howMuch;
time_t howMuch;
```

Обычно к псевдонимам typedef добавляют окончание `_t`, указывая, таким образом, что идентификатором является тип, а не переменная.

typedef не определяет новый тип данных. Это просто псевдоним (другое имя) для уже существующего типа. Его можно использовать везде, где используется обычный тип.

Даже если следующее не имеет смысла, оно все равно разрешено в языке C++:

```
typedef long miles_t;
typedef long speed_t;

miles_t distance = 8;
speed_t phr = 2100;

// Следующее разрешено, поскольку обе переменные distance и phr являются типа long
distance = phr;
```

typedef и читабельность кода

typedef используется в улучшении документации и разборчивости кода. Имена таких типов, как char, int, long, double и bool хороши для описания того, какой тип возвращает функция, но чаще всего мы хотим знать, с какой целью возвращается значение. Например, рассмотрим следующую функцию:

```
int GradeTest();
```

Мы видим, что возвращаемым значением является целое число, но что оно означает? Количество пропущенных вопросов? Идентификационный номер учащегося? Код ошибки? Сам int ни о чем нам не говорит. Исправим ситуацию:

```
typedef int testScore_t;
testScore_t GradeTest();
```

С использованием возвращаемого типа `testScore_t` становится очевидным, что функция возвращает тип, значением которого является результат теста.

typedef и поддержка кода

typedef также позволяет изменить базовый тип объекта без внесения изменений в большое количество кода. Например, если вы использовали тип short для хранения идентификационного номера учащегося, но потом решили, что лучше использовать тип long, то вам придется прошерстить кучу кода для замены short на long. И, вероятно, было бы трудно определить, какой из типов short используется для хранения идентификационных номеров, а какой — для других целей.

С typedef же всё, что вам нужно сделать, — это изменить объявление `typedef short studentID_t` на `typedef long studentID_t`. Тем не менее, не стоит забывать об осторожности при изменении типа typedef на тип из другого семейства (например, из int на float или наоборот)! Новый тип данных может иметь проблемы со сравнением или делением целых чисел/чисел **типа с плавающей точкой**, которых старый тип не имел — об этом следует помнить.

typedef и кроссплатформенность

Еще одним большим преимуществом typedef является возможность скрывать специфические для определенных платформ (операционных систем) детали. На некоторых платформах тип int занимает 2 байта, на других — 4 байта. Таким образом, использование типа int для хранения более 2 байтов информации может быть потенциально опасным при написании кроссплатформенного кода.

Поскольку char, short, int и long не указывают свой **размер**, то для кроссплатформенных программ довольно часто используется typedef для определения псевдонимов, которые включают размер типа данных в битах. Например, int8_t — это 8-битный **signed int**, int16_t — это 16-битный signed int, а int32_t — это 32-битный signed int.

typedef и упрощение сложного

Хотя мы до сих пор рассматривали только простые типы данных, в языке C++ вы можете увидеть и следующие переменные/функции:

```
std::vector<std::pair<std::string, int>> pairlist;

boolean hasAttribute(std::vector<std::pair<std::string, int>> pairlist)
{
    // Что-то делаем
}
```

Писать `std::vector<std::pair<std::string, int>>` всякий раз, когда нужно использовать этот тип — не очень эффективно и затратно как по времени, так и по приложенным усилиям. Гораздо проще использовать typedef:

```
typedef std::vector<std::pair<std::string, int>> pairlist_t; // используем pairlist_t в качестве псевдонима для этого длинного

pairlist_t pairlist; // объявляем pairlist_t

boolean hasAttribute(pairlist_t pairlist) // используем pairlist_t в качестве типа параметра функции
{
    // Что-то делаем
}
```

Вот! Другое дело! Ведь проще использовать `pairlist_t` вместо `std::vector<std::pair<std::string, int>>`, не так ли?

Не переживайте, если вы еще не знаете, что такое std::vector, std::pair и прочее. Гораздо важнее сейчас усвоить, что с помощью typedef вы можете давать простые имена сложным типам данных, что сделает их проще как для использования, так и для понимания.

type alias

У typedef есть также свои нюансы. Во-первых, легко забыть, что пишется первым: псевдоним типа или имя типа:

```
typedef time_t double; // неправильно
typedef double time_t; // правильно
```

Во-вторых, синтаксис typedef становится уже менее привлекательным в связке со сложными типами данных (об этом мы поговорим детально, когда будем рассматривать указатели на функции).

Для решения этих проблем, в C++11 ввели новый улучшенный синтаксис для typedef, который имитирует способ объявления переменных. Этот синтаксис называется **type alias**. С помощью type alias мы пишем имя, которое затем используется как синоним конкретного типа данных (т.е. принцип тот же, но синтаксис более удобен).

Следующий typedef:

```
typedef double time_t; // используем time_t в качестве псевдонима для типа double
```

В C++11 можно объявить как:

```
using time_t = double; // используем time_t в качестве псевдонима для типа double
```

Эти два способа функционально эквивалентны.

Обратите внимание, что хоть мы и используем ключевое слово `using`, оно не имеет ничего общего с **using-стейтментами**. Это ключевое слово имеет различный функционал в зависимости от контекста.

Новый синтаксис создания псевдонимов создает меньше проблем при использовании в сложных ситуациях, и его рекомендуется применять вместо обычного typedef, если ваш компилятор поддерживает C++11.

Правило: Используйте type alias вместо typedef, если ваш компилятор поддерживает C++11.

Тест

Задание №1

Используя следующий **прототип функции**:

```
int editData();
```

Преобразуйте тип возвращаемого значения int в status_t, используя ключевое слово typedef. В ответе к этому заданию укажите стейтмент typedef и обновленный прототип функции.

Задание №2

Используя прототип функции из задания №1, преобразуйте тип возвращаемого значения int в status_t, используя ключевое слово `using` (C++11). В ответе к этому заданию укажите стейтмент создания псевдонима типа и обновленный прототип функции.

Ответ №1

```
typedef int status_t;
status_t editData();
```

Ответ №2

```
using status_t = int;
status_t editData();
```